MDI3003 – ADVANCED PREDICTIVE ANALYTICS

Titanic Survival Prediction Using Machine Learning

GIHUB REPO LINK:

https://github.com/srijithramesh24/24MID0009_TITANIC-SURVIVAL-PREDICTION

SUBMITTED BY

SRIJITH R
24MID0009

UNDER GUIDANCE OF

Dr. DURGESH KUMAR
Assistant Professor Sr. Grade 1
VIT VELLORE

Abstract

This project develops a supervised machine-learning workflow to predict whether a passenger survived the Titanic disaster. The task is formulated as binary classification, where the target variable Survived takes the value 1 for a survivor and 0 for a non-survivor. The implementation uses the Kaggle Titanic training and test datasets and applies exploratory data analysis, feature selection, feature engineering, missing-value treatment, categorical encoding and numerical standardisation.

Three classifiers were implemented: Logistic Regression, Decision Tree and Random Forest. The notebook uses a stratified 80:20 split of the labelled training data, producing 712 training records and 179 validation records. Performance is measured using accuracy, precision, recall and F1-score. On the recorded validation split, Random Forest achieved the highest accuracy (81.56

Keywords

Titanic, survival prediction, supervised learning, binary classification, exploratory data analysis, Logistic Regression, Decision Tree, Random Forest, model evaluation.

Report scope

The report reflects the code and outputs present in the supplied executed notebook. It does not claim that the model establishes causal relationships or that validation performance guarantees performance on other populations.

1 Introduction

The Titanic survival prediction task is a widely used educational application of supervised machine learning. The historical dataset contains structured passenger records together with a recorded survival outcome. This project studies the available passenger attributes, prepares them for analysis, and applies classification algorithms to identify patterns associated with the recorded outcomes. The trained model is then used to generate predictions for passenger records whose outcomes are not provided.

The input information considered in the implementation includes passenger class, sex, age, number of siblings or spouses aboard, number of parents or children aboard, fare and port of embarkation. Two additional variables are engineered from family counts: FamilySize and IsAlone.

1.1 Motivation

The task is suitable for demonstrating data exploration, preprocessing and comparison of different classification approaches. It also makes it possible to discuss the difference between accuracy and other metrics, especially when false-positive and false-negative predictions have different interpretations.

1.2 Project boundary

This implementation is an educational benchmark using historical competition data. It is not a deployable emergency-response system, a causal analysis of survival, or a model validated for modern maritime incidents.

2 Objectives and Project Workflow

2.1 Objectives

1. Load and inspect the labelled training data and unlabelled test data.
2. Explore survival distribution and relationships with selected passenger attributes.
3. Identify missing values and prepare relevant features.
4. Engineer family-related features from existing passenger attributes.
5. Build a preprocessing pipeline for numerical and categorical variables.
6. Train and compare three supervised classification algorithms.
7. Evaluate predictions using multiple classification metrics and confusion matrices.
8. Select a model using validation F1-score and generate test-set predictions.

2.2 End-to-end workflow

Dataset loading → Dataset inspection → EDA → Feature preparation
↓
Preprocessing pipeline → Stratified split → Model training
↓
Validation and comparison → Model selection → Test predictions

2.3 Implementation environment

The supplied notebook is implemented in Python using Jupyter Notebook. The principal libraries are Pandas and NumPy for data handling, Matplotlib and Seaborn for visualisation, and Scikit-learn for preprocessing, classification and evaluation.

3 Dataset Description

The notebook loads three competition files: train.csv, test.csv and gender_submission.csv. The recorded dataset dimensions are shown below.

Table 1: Dataset dimensions recorded in the notebook

File	Rows	Columns
Training dataset	891	12
Test dataset	418	11
Gender submission example	418	2

3.1 Important attributes

Attribute	Description
PassengerId	Passenger identifier.
Survived	Target label: 0 for did not survive and 1 for survived. Pclass

	Passenger ticket class (1, 2 or 3).
Sex	Passenger gender category recorded in the dataset. Age
	Passenger age in years.
SibSp	Number of siblings or spouses aboard.
Parch	Number of parents or children aboard. Fare
	Passenger fare.
Embarked	Port of embarkation category.

The notebook selects seven initial predictors: Pclass, Sex, Age, SibSp, Parch, Fare and Embarked. The target column is Survived. The test file does not contain the target label.

3.2 Data preview

The executed notebook displays the first five training records and confirms 891 rows and 12 columns. The training data includes passenger identifiers, target labels and descriptive passenger fields.

4 Exploratory Data Analysis: Survival Distribution

Exploratory Data Analysis (EDA) is used to inspect the target distribution and identify patterns that may be useful during model development.

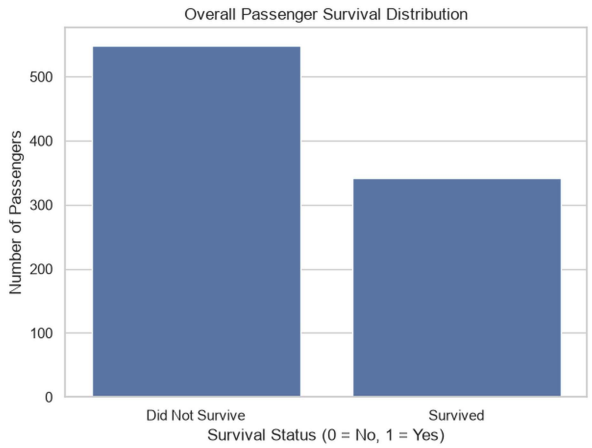


Figure 1: Distribution of the target variable in the training data.

The notebook records 549 non-survivors and 342 survivors. These counts correspond to approximately 61.62% and 38.38% of the 891 labelled passengers, respectively.

Table 2: Survival class counts

Target value	Meaning	Count
0	Did not survive	549
1	Survived	342

The target is not evenly distributed. This is one reason the project reports precision, recall and F1-score in addition to accuracy. A model that predicts the majority class frequently could obtain a seemingly reasonable accuracy while performing poorly on survivor identification.

5 Exploratory Data Analysis: Gender and Passenger Class

5.1 Survival by gender

The recorded survival percentages are 74.20% for female passengers and 18.89% for male passengers.

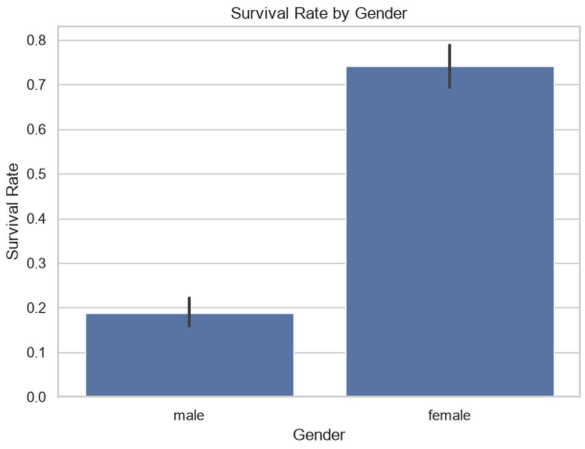


Figure 2: Survival rate by gender from the training dataset.

5.2 Survival by passenger class

The recorded survival rates are 62.96% for first class, 47.28% for second class and 24.24% for third class.

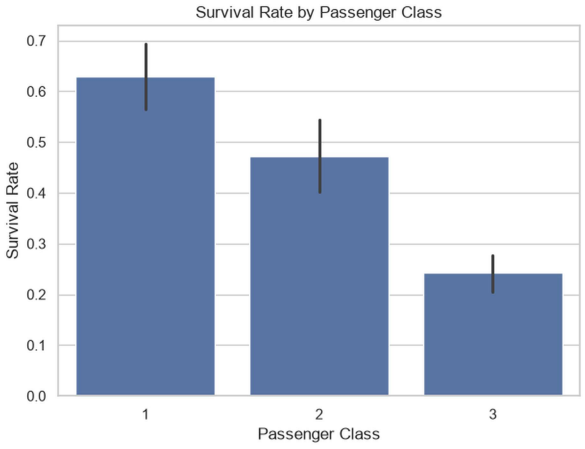


Figure 3: Survival rate by passenger class.

These are descriptive associations in the historical dataset. They should not be interpreted as proof that gender or ticket class alone caused an individual outcome. The model uses multiple features simultaneously, and the patterns are specific to this dataset.

6 Exploratory Data Analysis: Age, Missingness and Correlation

6.1 Age distribution

The notebook plots age distributions grouped by survival status. Age is a potentially informative feature, but its values are incomplete for a portion of the records.

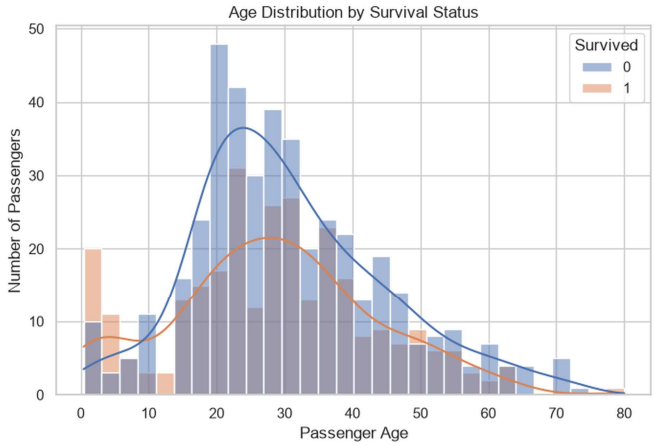


Figure 4: Age distribution grouped by recorded survival outcome.

6.2 Missing values

The notebook identifies missing values in Age, Cabin and Embarked.

Table 3: Missing values in the training dataset

Column	Missing count	Percentage
Age	177	19.87%
Cabin	687	77.10%
Embarked	2	0.22%

The selected model features do not include Cabin. Missing Age and Embarked values are handled inside the preprocessing pipeline.

6.3 Correlation analysis

A numerical-feature correlation heatmap is generated in the notebook. Correlation summarises linear association between numeric columns; it does not imply causation and does not directly describe relationships involving all categorical variables.

7 Feature Selection and Feature Engineering

The implementation selects seven original input features: Pclass, Sex, Age, SibSp, Parch, Fare and Embarked. The target vector is extracted separately from the Survived column.

7.1 Family-size feature

The notebook creates a derived feature:

$$\text{FamilySize} = \text{SibSp} + \text{Parch} + 1.$$

The constant 1 represents the passenger themselves. This combines two family-count attributes into a single feature.

7.2 Travelling-alone feature

A binary indicator is created:

$$\text{IsAlone} = \begin{cases} 1, & \text{if FamilySize} = 1, \\ 0, & \text{otherwise.} \end{cases}$$

For example, a passenger with SibSp = 0 and Parch = 0 has a family size of one and is marked as travelling alone. A passenger with one sibling aboard has a family size of two.

7.3 Final feature groups

The numerical features are Pclass, Age, SibSp, Parch, Fare, FamilySize and IsAlone. The categorical features are Sex and Embarked. The feature engineering is applied consistently to both the training and test feature tables.

7.4 Numerical relationships

The correlation heatmap provides a compact view of relationships among numerical attributes.



Figure 5: Correlation heatmap for numerical training attributes.

8 Preprocessing Pipeline

Data preprocessing is an essential stage of the Titanic Survival Prediction project. It converts raw passenger records into a suitable format for machine-learning algorithms by handling missing values, scaling numerical features and encoding categorical variables. The implementation uses Scikit-learn's ColumnTransformer to apply separate transformations to numerical and categorical features, combined with a Pipeline for model training.

8.1 Numerical transformation

The numerical features include Pclass, Age, SibSp, Parch, Fare, FamilySize and IsAlone. Missing numerical values are replaced using median imputation, which reduces the influence of extreme values. StandardScaler is then applied to bring numerical features to a common scale using the formula:

$$z = (x - \mu) / \sigma$$

where x represents the original value, μ represents the mean and σ represents the standard deviation calculated from the training data. This scaling is particularly useful for algorithms such as Logistic Regression.

8.2 Categorical transformation

The categorical features are Sex and Embarked. Missing categorical values are replaced using the most frequent category through SimpleImputer. OneHotEncoder converts categorical values into binary numerical columns. The parameter handle_unknown='ignore' ensures that previously unseen categories do not cause encoding errors during prediction.

8.3 Pipeline Integration and Data Leakage Prevention

The ColumnTransformer combines numerical and categorical transformations into a single preprocessing stage. This is integrated with each classification algorithm using Scikit-learn's Pipeline. The same transformations are consistently applied during training, validation and testing. To prevent data leakage, imputation values, scaling parameters and category information are learned only from the training partition. The validation data is transformed using these learned parameters without influencing the fitting process.

8.4 Stratified Train-Validation Split

The labelled dataset is divided using train_test_split() with an 80:20 ratio, random_state=42 and stratification based on the Survived target variable.

- **Training Dataset:** 712 records (439 non-survivors and 273 survivors).
- **Validation Dataset:** 179 records (110 non-survivors and 69 survivors).
- Stratification maintains approximately the same survival-class distribution in both partitions. The training dataset is used to fit the classification models, while the validation dataset is used to evaluate and compare their performance.
- Overall, the preprocessing pipeline ensures consistent data preparation, handles missing information, converts categorical attributes into numerical representations and prepares the Titanic dataset for machine-learning classification and survival prediction.

9 Classification Models

This kaggle project implements three supervised machine-learning classification algorithms: Logistic Regression, Decision Tree and Random Forest. All three models use the same preprocessing pipeline and stratified validation dataset to ensure a fair comparison. The models learn patterns from passenger attributes such as age, gender, passenger class, fare and family information to predict whether a passenger survived the Titanic disaster.

9.1 Logistic Regression

Logistic Regression is a supervised classification algorithm that estimates the probability of a passenger belonging to the positive class (survived). It uses the sigmoid function to convert a linear combination of input features into a probability between 0 and 1.

$$P(Y = 1 \mid x) = \frac{1}{1 + e^{-(\beta_0 + \beta_1 x)}}$$

The predicted class is determined using the default decision threshold of 0.5. If the predicted probability is greater than or equal to 0.5, the passenger is classified as a survivor; otherwise, the passenger is classified as a non-survivor. The notebook uses max_iter=1000 to allow sufficient iterations for model convergence.

9.2 Decision Tree

Decision Tree is a supervised learning algorithm that makes predictions through a sequence of feature-based decisions. It divides the dataset into smaller groups using conditions on input features until a prediction is reached at a leaf node. For example, the model can learn decision rules involving passenger class, gender and age. The implementation uses max_depth=5 and random_state=42. Limiting the maximum depth helps control model complexity and reduces the possibility of overfitting.

9.3 Random Forest

Random Forest is an ensemble learning algorithm that combines multiple decision trees to improve prediction performance. Each tree is trained using a different sample of the training data, and their predictions are combined through majority voting for classification. This approach generally provides greater stability than relying on a single decision tree. The notebook uses n_estimators=200, max_depth=6 and random_state=42.

9.4 Training procedure

Each classifier is integrated with the preprocessing pipeline and trained using the 712 training records. The trained models then predict survival outcomes for the 179 validation records. Their performance is compared using accuracy, precision, recall and F1-score. The model with the highest validation F1-score is selected for the final prediction stage. This common training and evaluation procedure ensures a consistent comparison of all three classification algorithms.

10 Evaluation Metrics

Model evaluation is an important stage of the Titanic Survival Prediction project. It measures how accurately the trained classifiers predict passenger survival and helps identify the strengths and weaknesses of each model. The project evaluates Logistic Regression, Decision Tree and Random Forest using four primary metrics: Accuracy, Precision, Recall and F1-score. These metrics are calculated using the confusion matrix, which consists of four prediction outcomes.

- True Positive (TP): Passenger actually survived and was correctly predicted as a survivor.
- True Negative (TN): Passenger did not survive and was correctly predicted as a non-survivor.
- False Positive (FP): Passenger did not survive but was incorrectly predicted as a survivor.
- False Negative (FN): Passenger survived but was incorrectly predicted as a non-survivor.

10.1 Accuracy

Accuracy measures the proportion of correctly classified passengers among all validation records.

$$\text{Accuracy} = \frac{TP + TN}{TP + TN + FP + FN}$$

Type equation here.

A higher accuracy indicates that the classifier correctly predicts a larger proportion of the validation records.

10.2 Precision

Precision measures the proportion of predicted survivors who were actually survivors.

$$Precision = \frac{TP}{TP + FP}$$

Higher precision indicates fewer false-positive predictions.

10.3 Recall

Recall measures the proportion of actual survivors correctly identified by the classifier.

$$Recall = \frac{TP}{TP + FN}$$

Higher recall indicates that the model identifies more of the actual survivors.

10.4 F1-score

F1-score is the harmonic mean of precision and recall.

$$F1 = 2 \times \frac{Precision \times Recall}{Precision + Recall}$$

It provides a balance between precision and recall. The notebook uses F1-score as the model-selection criterion to identify the best-performing classifier.

10.5 Confusion matrix

A confusion matrix provides a detailed comparison between actual and predicted class labels. It displays the counts of true negatives, false positives, false negatives and true positives. It helps identify the types of classification errors made by each model and provides a clearer understanding of performance beyond overall accuracy.

11 Experimental Results and Model Comparison

The following values are taken directly from the executed notebook's validation output.

Table 4: Validation performance of the three classifiers

Model	Accuracy	Precision	Recall	F1-score
Logistic Regression	0.8045	0.7833	0.6812	0.7287
Decision Tree	0.7654	0.7547	0.5797	0.6557
Random Forest	0.8156	0.8750	0.6087	0.7179

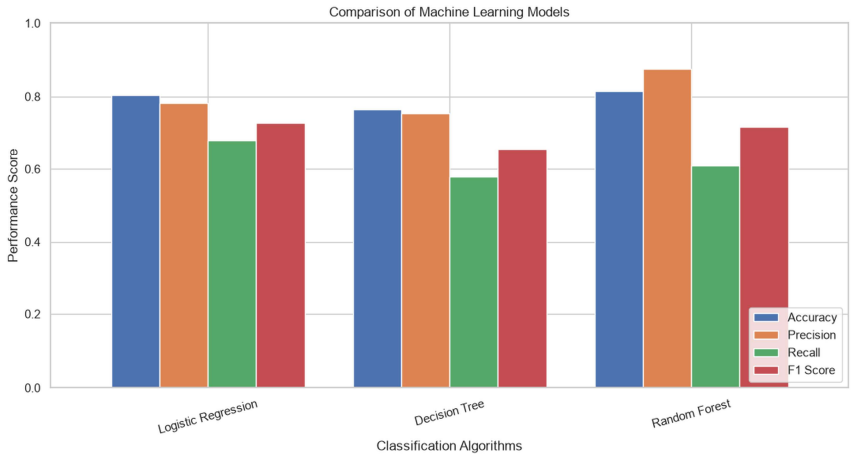


Figure 6: Comparison of validation accuracy, precision, recall and F1-score.

11.1 Interpretation

The validation results indicate that Random Forest achieved the highest accuracy of 81.56% and precision of 87.50% among the three classifiers. This shows that it provided the highest overall classification accuracy and was highly reliable when predicting passenger survival. However, its recall was 60.87%, meaning that it identified approximately 61% of the passengers who actually survived.

Logistic Regression achieved the highest recall of 68.12% and F1-score of 72.87%, with an accuracy of 80.45%. Its higher recall indicates that it identified a greater proportion of actual survivors compared with the other models. The highest F1-score demonstrates a better balance between precision and recall, making Logistic Regression a strong candidate for the final prediction model. Decision Tree recorded the lowest performance, with an accuracy of 76.54% and an F1-score of 65.57%.

Based on the comparison, Logistic Regression was selected as the final model because it achieved the highest F1-score. Although Random Forest performed better in accuracy and precision, Logistic Regression provided a better balance between identifying actual survivors and maintaining prediction reliability. This comparison highlights the importance of considering multiple evaluation metrics rather than relying only on accuracy when selecting a machine learning classifier.

12 Confusion Matrix Analysis and Model Selection

The notebook generates a confusion matrix for each classifier using the same validation labels and predictions.

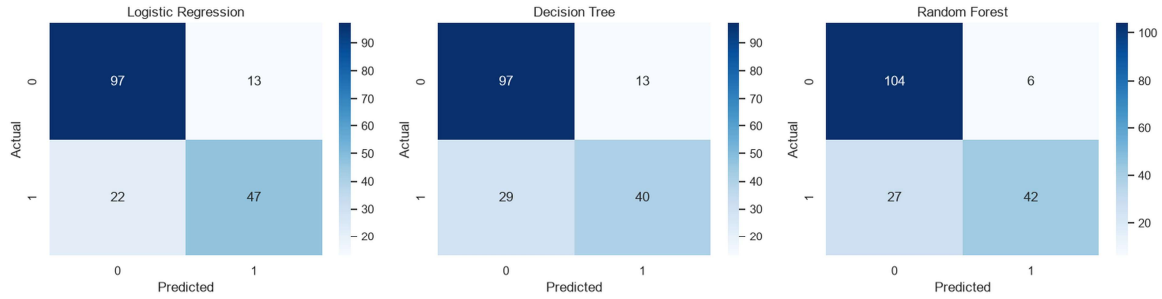


Figure 7: Confusion matrices for Logistic Regression, Decision Tree and Random Forest.

12.1 Best model selection

The code I run selects the model with the maximum validation F1-score. The recorded output is:

Selected classifier: Logistic Regression
Validation F1-score: **0.7287** (72.87%)

Although Random Forest has a higher accuracy, Logistic Regression is selected because the implemented selection criterion is F1-score. This is consistent with the stated model-selection procedure.

12.2 Limitations of the experiment

The experimental results are based on a single stratified 80:20 training-validation split with a fixed random seed of 42. Although stratification maintains a similar proportion of survivors and non-survivors in both datasets, the performance scores may change if a different data split is used. Therefore, the reported results represent the models' performance on the selected validation dataset rather than their guaranteed performance on all unseen data.

Additionally, the experiment uses predefined model parameters without performing systematic hyperparameter tuning or repeated cross-validation. Techniques such as Grid Search, Random Search, and k-fold cross-validation could improve model optimization and provide a more reliable estimate of generalization performance. The dataset also contains missing values, particularly in the Age and Cabin columns, which may limit the information available for prediction.

Finally, Logistic Regression was selected based on its highest validation F1-score of 72.87%. However, this does not establish that it is the universally best classifier for Titanic survival prediction. Further evaluation using multiple validation splits, optimized model parameters, and additional feature engineering would be necessary to confirm the model's stability and improve its predictive performance.

13 Final Prediction, Conclusion and Future Work

13.1 Final prediction stage

After model selection, the notebook refits the Logistic Regression pipeline on all 891 labelled training records. It then predicts the 418 records in the competition test dataset and creates a CSV file with two columns: PassengerId and Survived.

The recorded prediction distribution is 259 non-survivors (class 0) and 159 survivors (class 1). The first rows displayed by the notebook include passenger IDs 892–896. These are model predictions for the unlabelled test data, not verified ground-truth outcomes.

13.2 Conclusion

The project implemented an end-to-end supervised classification workflow for Titanic passenger survival prediction. It covered dataset inspection, exploratory visualisation, feature engineering, missing-value handling, categorical encoding, scaling, stratified validation, model training and evaluation. Among the three tested classifiers, Logistic Regression achieved the highest validation F1-score of 72.87% and was selected by the notebook's F1-based selection rule. The final fitted pipeline generated 418 test-set predictions in the Kaggle submission format.

13.3 Future improvements

- Evaluate the models using stratified cross-validation to obtain more robust performance estimates.
- Explore additional features such as title extraction from passenger names, cabin deck and ticket-group information.
- Perform systematic hyperparameter tuning using a validation strategy.
- Examine class-specific errors and probability calibration.
- Compare the local validation results with the Kaggle leaderboard score after submission.

13.4 References

1. Kaggle, "Titanic – Machine Learning from Disaster," https://www.kaggle.com/competitions/ titanic.
2. Scikit-learn Developers, "scikit-learn: Machine Learning in Python," https://scikit-learn. org/stable/.
3. Scikit-learn Developers, "Model evaluation: quantifying the quality of predictions," https:// scikit-learn.org/stable/modules/model_evaluation.html.
4. Pandas Development Team, "pandas documentation," https://pandas.pydata.org/docs/.

APPENDIX A

This appendix presents the important Python code used for dataset preparation, feature engineering, preprocessing, classification, model evaluation and final prediction.

A Dataset Preparation

The dataset is loaded using Pandas. Family-related features are created to improve the classification process.

```
import pandas as pd
import numpy as np

train = pd.read_csv("train.csv")
test = pd.read_csv("test.csv")

for df in [train, test]:
    df["FamilySize"] = df["SibSp"] + df["Parch"] + 1
    df["IsAlone"] = (df["FamilySize"] == 1).astype(int)

features = [
    "Pclass", "Sex", "Age", "SibSp",
    "Parch", "Fare", "Embarked",
    "FamilySize", "IsAlone"
]

X = train[features]
y = train["Survived"]
X_test = test[features]
```

B Data Preprocessing

Numerical and categorical features are transformed using separate preprocessing pipelines.

```
from sklearn.pipeline import Pipeline
from sklearn.compose import ColumnTransformer
from sklearn.impute import SimpleImputer
from sklearn.preprocessing import StandardScaler, OneHotEncoder

numeric_features = [
    "Pclass", "Age", "SibSp", "Parch",
    "Fare", "FamilySize", "IsAlone"
]

categorical_features = ["Sex", "Embarked"]

numeric_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="median")),
    ("scaler", StandardScaler())
])

categorical_pipeline = Pipeline([
    ("imputer", SimpleImputer(strategy="most_frequent")),
    ("encoder", OneHotEncoder(handle_unknown="ignore"))
])
```

```
preprocessor = ColumnTransformer([
    ("num", numeric_pipeline, numeric_features),
    ("cat", categorical_pipeline, categorical_features)
])
```

C Training and Validation Split

```
from sklearn.model_selection import train_test_split

X_train, X_val, y_train, y_val = train_test_split(
    X, y,
    test_size=0.20,
    random_state=42,
    stratify=y
)

print("Training records:", X_train.shape[0])
print("Validation records:", X_val.shape[0])
```

D Classification Model Development

Three classification algorithms are configured and compared using the same dataset.

```
from sklearn.linear_model import LogisticRegression
from sklearn.tree import DecisionTreeClassifier
from sklearn.ensemble import RandomForestClassifier

models = {
    "Logistic Regression":
        LogisticRegression(max_iter=1000),

    "Decision Tree":
        DecisionTreeClassifier(
            max_depth=5,
            random_state=42
        ),

    "Random Forest":
        RandomForestClassifier(
            n_estimators=200,
            max_depth=6,
            random_state=42
        )
}
```

E Model Training and Evaluation

Each classifier is trained and evaluated using accuracy, precision, recall and F1-score.

```
from sklearn.metrics import (
    accuracy_score,
    precision_score,
    recall_score,
    f1_score
```

```
)

results = []
trained_models = {}

for name, classifier in models.items():

    pipeline = Pipeline([
        ("preprocessor", preprocessor),
        ("classifier", classifier)
    ])

    pipeline.fit(X_train, y_train)
    predictions = pipeline.predict(X_val)

    trained_models[name] = pipeline

    results.append({
        "Model": name,
        "Accuracy": accuracy_score(y_val, predictions),
        "Precision": precision_score(y_val, predictions),
        "Recall": recall_score(y_val, predictions),
        "F1-Score": f1_score(y_val, predictions)
    })

results_df = pd.DataFrame(results)
print(results_df)
```

F Confusion Matrix

The confusion matrix is generated to examine correct and incorrect classifications.

```
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix

fig, axes = plt.subplots(1, 3, figsize=(12, 4))

for ax, (name, model) in zip(
    axes, trained_models.items()
):
    predictions = model.predict(X_val)
    cm = confusion_matrix(y_val, predictions)

    sns.heatmap(
        cm,
        annot=True,
        fmt="d",
        cmap="Greys",
        ax=ax
    )

    ax.set_title(name)
    ax.set_xlabel("Predicted")
    ax.set_ylabel("Actual")

plt.tight_layout()
plt.show()
```

G Model Selection

The classifier with the highest validation F1-score is selected as the preferred model.

```
best_index = results_df["F1-Score"].idxmax()

best_model_name = results_df.loc[
    best_index, "Model"
]

best_model = trained_models[best_model_name]

print("Selected Model:", best_model_name)
print("Best F1-Score:",
      results_df.loc[best_index, "F1-Score"])
```

H Final Model Training

Logistic Regression is retrained using the complete labelled training dataset, based on the recorded model-selection result.

```
final_model = Pipeline([
    ("preprocessor", preprocessor),
    ("classifier", LogisticRegression(max_iter=1000))
])

final_model.fit(X, y)
```

I Final Test Prediction

The trained model generates survival predictions for the passenger records in the test dataset.

```
test_predictions = final_model.predict(X_test)

submission = pd.DataFrame({
    "PassengerId": test["PassengerId"],
    "Survived": test_predictions
})

submission.to_csv(
    "titanic_submission.csv",
    index=False
)

print(submission.head())
```

J Final Output Verification

```
print("Submission Shape:", submission.shape)

print("Prediction Distribution:")
print(submission["Survived"].value_counts())

print("Total Predictions:", len(submission))
```

K Implementation Summary

The implementation covers dataset preparation, feature engineering, preprocessing, classification, model evaluation and final prediction generation.

Logistic Regression was selected with a validation F1-score of 72.87%. The final prediction file contains 418 passenger records, including 259 predicted non-survivors and 159 predicted survivors.

END OF APPENDIX

GIHUB REPO LINK:

https://github.com/srijithramesh24/24MID0009_TITANIC-SURVIVAL-PREDICTION